

CG_HW2_ImageWarping 实验报告

李亭皓 PB23010374

2026 年 3 月 15 日

1 问题背景

图像变形 (image warping) 的基本问题可以表述为：给定若干对控制点

$$(\mathbf{p}_i, \mathbf{q}_i), \quad i = 1, 2, \dots, n,$$

其中 $\mathbf{p}_i \in \mathbb{R}^2$ 为原图中的源点， $\mathbf{q}_i \in \mathbb{R}^2$ 为用户期望它移动到的目标点，构造一个二维到二维的映射

$$f: \mathbb{R}^2 \rightarrow \mathbb{R}^2,$$

使其满足插值条件

$$f(\mathbf{p}_i) = \mathbf{q}_i, \quad i = 1, 2, \dots, n.$$

随后再利用该映射对整张图像进行重采样，从而得到变形结果。

本次作业在课程提供的 **Framework2D** 框架上进行。框架已经实现了图像读写、基础图像处理、交互选点以及一个简单的鱼眼变形示例，而作业的核心要求是补充两类典型的散乱数据插值方法：**IDW** (Inverse Distance Weighted interpolation) 与 **RBF** (Radial Basis Function interpolation)，并将 warping 的数学变换与图像遍历过程解耦，封装为独立的 warper 类。

如果只从“满足插值条件”这一点来看，问题本身已经比较明确：需要一种从少量控制点对出发，生成整张平面连续形变场的方法。但在实际交互中，我更关注的并不只是“点有没有精确到位”，还包括以下几个更贴近编辑体验的问题：

1. 变形是否足够局部，是否会明显带动远处无关区域；
2. 形变是否平滑，是否会出现割裂、突变或明显不自然的压缩；
3. 是否需要大量控制点才能产生理想效果；
4. 方法是否更接近常见液化 (liquify) 工具的使用体验，即少量操作即可推动一整片区域自然变形。

因此，在完成课程要求的基础上，本文还尝试将本项目理解为一面向**液化式局部编辑**的图像变形实验。

具体来说，我希望在尽量保持实现简洁的前提下，增强局部性，减少对“固定点”与密集控制点的依赖，使少量控制点也能对连续轮廓产生较自然的整体影响。

2 程序使用说明

本实验实现的是一个基于 `Framework2D + ImGui` 的图像变形程序。程序启动后，界面主要由顶部主菜单栏、其下方的算法工具栏，以及中央的图像显示区域组成。用户的一般流程为：打开图像、选择算法、录入控制点对、执行 warping、必要时补洞、最后保存或恢复。

2.1 界面组成

2.1.1 主菜单栏

主菜单栏包含以下功能：

- **File → Open Image File..**: 打开图像文件；
- **File → Save As..**: 保存当前结果；
- **Invert**: 颜色反相；
- **Mirror**: 镜像；
- **GrayScale**: 灰度化；
- **Select Points**: 进入控制点对选择模式；
- **Warping**: 执行当前算法对应的图像变形；
- **Restore**: 恢复到最初载入的原始图像。

其中，**Invert**、**Mirror** 与 **GrayScale** 属于基础图像操作；**Select Points**、**Warping** 与 **Restore** 构成了图像变形的主要交互流程。

2.1.2 算法工具栏

算法工具栏用于选择 warping 方法。当前程序支持：

- **Fisheye**
- **IDW**
- **IDW__A**
- **IDW__R**
- **RBF__AffineFirst**
- **RBF__Full**

此外，界面中还提供 **Fill Holes** 按钮，用于在前向映射后对结果中的空洞区域做补全。当前补洞实现采用 ANN 风格的局部近邻插值。

2.2 基本操作流程

2.2.1 打开图像

点击 **Open Image File..** 后，在对话框中选择待处理图像。程序支持打开常见的 .png 与 .jpg 文件。打开成功后，图像会显示在编辑区域中。

程序在载入图像时会自动保存一份原始备份，因此之后即使进行多次 warping 和补洞，也可以通过 **Restore** 回到初始状态。

2.2.2 选择控制点对

点击 **Select Points** 后，程序进入控制点对输入模式。用户在图像区域中进行如下操作：

1. 在某一点按下鼠标左键，记录起点；
2. 拖动鼠标时，界面实时显示从起点指向当前鼠标位置的红色线段；
3. 松开鼠标后，当前位置被记录为终点，从而形成一组完整控制点对。

每组控制点对都会以可视化形式显示：

- 蓝色圆点表示起点；
- 绿色圆点表示终点；
- 红色线段表示位移方向和大小。

这种交互模式对应的是“拖动式指定一对点”，而不是“两次单击分别选起点和终点”。

2.2.3 执行变形

选择算法并录入控制点对后，点击 **Warping**，程序将根据当前算法类型构造对应的 warper，并对整张图像执行一次前向映射变形。执行完成后：

- 图像内容更新为变形结果；
- 控制点对列表被清空；
- 程序保留一份当前 warping 结果的备份，供后续补洞使用。

由于当前实现采用前向映射，目标图像上可能会出现未被写入像素的空洞区域。这些空洞可以通过后续的 **Fill Holes** 处理。

2.2.4 补洞、恢复与保存

若变形结果中出现明显空洞，可点击 **Fill Holes**。程序将基于已写入像素与理论覆盖区域进行受限补洞。若结果不满意，可点击 **Restore** 恢复初始图像。若结果满意，则可通过 **Save As..** 保存当前图像。

2.3 小结

整体而言，程序的使用逻辑为：

打开图像 → 选择算法 → 录入控制点对 → 执行 warping → 必要时补洞 → 保存或恢复。

这种组织方式足以支撑本实验中对多种 warping 方法的直观比较。

3 系统设计与代码结构

3.1 Warper 基类的抽象

按照课程要求，warping 的数学变换不应直接写死在图像处理流程中，而应被抽象成与图像内容无关的独立类。本文将这一部分封装为 Warper 基类，其核心职责包括：

- 统一保存控制点对与画布尺寸；
- 对外提供统一的 `warp(const ImVec2& p)` 接口；
- 处理空控制点与精确控制点插值等通用情况；
- 将实际算法实现留给派生类的 `warp_impl()`。

这一设计带来的直接好处是：图像层代码只需关心“遍历像素并调用 `warp()`”，而不需要知道当前内部使用的是 IDW、RBF 还是其他变形算法。

3.2 整体调用关系

本文的实现大致可以分为三层：

1. **界面层**：负责按钮、算法选择、控制点输入与结果显示；
2. **warper 层**：负责几何映射公式的建立与查询；
3. **图像层**：负责遍历原图像素，将其按 warper 给出的目标位置写入结果图。

在执行 Warping 时，程序首先根据界面当前的算法按钮创建相应的 warper 对象，例如 IDWWarper 或 RBFWarper，随后遍历原图像素，对每个点 (x, y) 调用

$$(x', y') = f(x, y)$$

得到目标位置，并将颜色写入结果图。

从结构上看，这种组织方式比较清晰：控制逻辑、数学映射和图像搬运彼此分离，也便于后续增加新的 warping 方法。

4 IDW 方法的实现与改进

4.1 原始 IDW 的基本逻辑

IDW 的基本思想是：对每个控制点构造一个局部映射，再根据查询点到各控制点的距离，对这些局部映射结果做归一化加权平均。本文实现中，每个控制点 i 对应一个局部仿射映射

$$f_i(\mathbf{p}) = \mathbf{q}_i + \mathbf{T}_i(\mathbf{p} - \mathbf{p}_i),$$

其中 $\mathbf{T}_i$ 是一个 2×2 矩阵，用于描述控制点附近的局部线性变换趋势。

对于查询点 $\mathbf{p}$ ，原始 IDW 的结果写为

$$f(\mathbf{p}) = \frac{\sum_{i=1}^n w_i(\mathbf{p}) f_i(\mathbf{p})}{\sum_{i=1}^n w_i(\mathbf{p})},$$

其中本文采用的基本权重为

$$w_i(\mathbf{p}) \propto \frac{1}{\|\mathbf{p} - \mathbf{p}_i\|^2}.$$

在代码中，这一部分对应 `control_weight()` 与 `Type::IDW` 分支。为避免距离过小带来的数值问题，分母中加入了很小的 ε 作保护。

4.2 局部仿射矩阵的求解

相比简单地对位移向量直接插值，本文仍保留了 IDW 文档中的局部仿射思想。对每个控制点 i ，利用其他控制点 $j \neq i$ 的相对位置与目标变化，构造局部最小二乘问题，希望满足

$$\mathbf{T}_i(\mathbf{p}_j - \mathbf{p}_i) \approx \mathbf{q}_j - \mathbf{q}_i.$$

写成矩阵形式，可得到

$$\mathbf{A} = \sum_{j \neq i} w_{ij} (\Delta \mathbf{p}_{ij}) (\Delta \mathbf{p}_{ij})^\top, \quad \mathbf{B} = \sum_{j \neq i} w_{ij} (\Delta \mathbf{q}_{ij}) (\Delta \mathbf{p}_{ij})^\top,$$

并令

$$\mathbf{T}_i = \mathbf{B} \mathbf{A}^{-1}.$$

实现中，若邻域信息不足或矩阵求解失败，则退化为单位矩阵。这种写法虽然简单，但能够较自然地表达控制点周围的局部变换趋势。

4.3 原始 IDW 的效果与不足

实验中，原始 IDW 的总体效果是可以接受的：它实现简单，参数不多，也较容易与交互界面结合。但在实际使用中，我逐渐注意到以下几个问题：

1. 当控制点只有一个，或者多个控制点近似共线时，可用于拟合局部仿射的信息较少，变形自由度比较有限；
2. 原始 IDW 虽然具有一定局部性，但其影响并非真正有限支撑，远处点仍会受到拖动，因此若想让某些区域保持基本不动，往往需要额外设置边缘控制点；
3. 对于一条连续图像轮廓，仅靠少量控制点通常不足以让整段曲线自然一起变形，常常需要较密集地取点，否则容易出现局部扭曲。

这一观察与本文的额外目标是相关的：如果希望更接近液化工具的交互方式，那么单纯满足控制点插值还不够，还需要进一步提高局部控制的自然性。

4.4 IDW_R：引入有限作用半径

为增强局部性，本文实现了一个紧支撑版本 IDW_R。它对每个控制点引入一个有限作用半径 R_i ，仅在

$$\|\mathbf{p} - \mathbf{p}_i\| < R_i$$

时赋予非零权重。对应权重函数写为

$$w_i(\mathbf{p}) = \left(\frac{R_i - \|\mathbf{p} - \mathbf{p}_i\|}{R_i} \right)^\mu, \quad \|\mathbf{p} - \mathbf{p}_i\| < R_i,$$

否则取 0。

这一做法确实增强了局部性，但也带来了新的问题。实验中可以观察到，若两个控制点的作用范围彼此不相交，则中间区域几乎不发生形变；而若两个作用域仅在局部重叠，则重叠区域的变化又可能偏强，整张图像容易出现分块感。换句话说，问题并不完全在于核函数是否足够平滑，更在于固定控制边界本身会引入人工的作用范围分割。

4.5 从“改核函数”到“改位移传播”的思路转变

在发现 IDW_R 仍不理想后，我起初将主要问题归结为“核函数不够平滑”，因此尝试过若干不同的 $\sigma(r)$ 形式，希望同时满足：

- 控制点中心权重较高；
- 衰减尾部较平滑；
- 尽量保留一定重尾性；

- 到零处不要出现过于生硬的视觉边界。

实验中尝试过的权重形式包括：经典负幂、加平滑常数的幂函数、高斯型核、紧支撑核，以及一个自定义版本

$$\sigma(r) = \frac{1}{\sqrt{r^2 + c^2}} \cdot \frac{1}{1 + r^2/a^2}, \quad c = 0.1, a = 10.$$

这些修改确实会影响控制点之间的加权过渡，但后来我逐渐意识到：它们主要改变的是“点与点之间如何竞争”，却没有从根本上解决“如何让一整段区域用较少控制点自然一起运动”的问题。以瘦脸为例，用户通常并不希望脸部只在某个锚点附近凹进去一小块，而是希望整段轮廓向内收，这更像是一个区域性位移传播问题，而不只是权重曲线的问题。

4.6 IDW_A：对局部位移引入径向衰减场

在上述思路基础上，本文最终实现了 IDW_A。与其继续单独修改距离核函数，不如直接对每个控制点诱导出的**局部位移**做衰减。

设

$$f_i(\mathbf{p}) = \mathbf{q}_i + \mathbf{T}_i(\mathbf{p} - \mathbf{p}_i),$$

则控制点 i 对当前点给出的局部位移建议可写为

$$\Delta_i(\mathbf{p}) = f_i(\mathbf{p}) - \mathbf{p}.$$

本文为其乘上一个径向衰减函数

$$A_i(\mathbf{p}) = \left(1 - \frac{\|\mathbf{p} - \mathbf{p}_i\|^2}{R_i^2}\right)^2, \quad \|\mathbf{p} - \mathbf{p}_i\| < R_i,$$

在半径外则取 0。于是最终映射写成

$$f(\mathbf{p}) = \mathbf{p} + \frac{\sum_i w_i(\mathbf{p}) A_i(\mathbf{p}) \Delta_i(\mathbf{p})}{\sum_i w_i(\mathbf{p})}.$$

这里仍保留原始 IDW 的简单负幂权重 w_i ，但把每个控制点的位移幅度单独做了径向衰减。这样的分工比较清晰：

- w_i 决定当前点更相信哪个控制点的局部模型；
- A_i 决定该模型的位移影响向外传播到什么程度。

从实验结果看，这种写法比单纯修改 $\sigma(r)$ 更贴近我的目标，因为我真正希望控制的是**局部位移场的传播方式**，而不仅仅是权重竞争本身。

4.7 作用半径的设计

无论在 IDW_R 还是 IDW_A 中，作用半径 R_i 都不宜设为固定常数，而应与用户本次拖动的位移长度相关。为此，本文采用

$$R_i = \max\{R_{\min}, \alpha\|\mathbf{q}_i - \mathbf{p}_i\| + \beta\},$$

其中

$$\alpha = 1.7, \quad \beta = 8, \quad R_{\min} = 16.$$

这一设计的考虑是：若用户拖动较小，则作用范围不应退化得过小；若拖动较大，则影响范围也应相应扩大，以留出足够缓冲区来完成平滑过渡。参数 $\alpha = 1.7$ 并非严格推导所得，而是结合实验观察得到的一个经验值。它的作用主要是让作用半径明显大于实际位移长度，减小局部压缩过强、进而产生翻折的风险。

需要说明的是，这一参数选择仍有进一步调节空间。若后续时间更充分，仍可以继续尝试不同的 R_i 参数、不同的衰减函数 A_i ，甚至更一般的各向异性作用域。

4.8 小结

本文实现的三种 IDW 方法可以概括如下：

- IDW：原始全局 IDW，结构简单，但存在一定全局牵动；
- IDW_R：通过紧支撑增强局部性，但容易引入分块感与人工边界；
- IDW_A：保留原始距离权重，在局部位移上引入衰减场，更接近“液化式局部传播”的需求。

就本文当前实验而言，IDW_A 是最符合我预期的一种折中：它没有明显增加实现复杂度，但在局部性与连续性之间提供了相对更稳定的结果。

5 RBF 方法的实现

5.1 RBF 的统一模型

与 IDW 不同，RBF 方法不是在查询时动态做距离加权，而是先假设整个变形场具有“仿射部分 + 径向部分”的统一形式：

$$f(\mathbf{p}) = A(\mathbf{p}) + R(\mathbf{p}),$$

其中

$$A(\mathbf{p}) = A\mathbf{p} + \mathbf{b}$$

表示全局仿射部分，

$$R(\mathbf{p}) = \sum_{i=1}^n \alpha_i g_i(\|\mathbf{p} - \mathbf{p}_i\|)$$

表示由控制点诱导出的径向补偿项。这里 $\alpha_i \in \mathbb{R}^2$ 为待求系数。

因此，RBF 的核心工作在于：先根据控制点对求出仿射矩阵 $\mathbf{A}$ 、平移 $\mathbf{b}$ 以及所有径向系数 α_i ，随后对任意查询点直接代入统一公式即可。

5.2 基函数与局部尺度

本文代码中为每个控制点设置了一个局部尺度

$$r_i = \min_{j \neq i} \|\mathbf{p}_i - \mathbf{p}_j\|,$$

即该点到其他控制点的最近距离。随后基函数取为

$$g_i(d) = \sqrt{d^2 + r_i^2}$$

或

$$g_i(d) = \frac{1}{\sqrt{d^2 + r_i^2}},$$

当前默认使用后者。

这种写法的优点在于：不同控制点的基函数尺度能够随点分布自动调整，不必额外手动指定一个全局统一半径。

5.3 两种求解方式

本文实现了两类 RBF 接口：

- **RBF__AffineFirst**
- **RBF__Full**

需要说明的是，这两者并不是两种完全不同的模型，而是同一 RBF 模型的两种求解方式。

5.3.1 AffineFirst

AffineFirst 的思路是：先拟合仿射部分，再让径向部分去解释剩余误差。

具体实现中：

- 若只有一个控制点，则仿射部分退化为平移；
- 若有两个控制点，则采用平移加各向同性缩放；
- 若控制点不少于三个，则对

$$\min \sum_i \|\mathbf{A}\mathbf{p}_i + \mathbf{b} - \mathbf{q}_i\|^2$$

做最小二乘拟合，求出仿射参数。

在仿射部分确定后，计算每个控制点上的残差

$$\mathbf{r}_j = \mathbf{q}_j - (A\mathbf{p}_j + \mathbf{b}),$$

再建立线性系统

$$G_{j,i} = g_i(\|\mathbf{p}_j - \mathbf{p}_i\|),$$

分别求出 x, y 两个分量上的径向系数。

这种方法的优点是结构清楚：先用低维全局模型解释大趋势，再让 RBF 只负责补偿非仿射误差。

5.3.2 Full

Full 的思路则是：把仿射项与径向系数一起放入同一个更大的线性系统中联合求解。

它在插值条件

$$f(\mathbf{p}_j) = \mathbf{q}_j$$

之外，还加入若干附加约束，用来分离仿射部分与径向部分，避免表示不唯一。求解完成后，再从线性系统的解向量中恢复出所有 α_i 以及仿射参数。

从实现角度看，Full 与 AffineFirst 的差异主要不在最终查询公式，而在模型参数建立阶段。

5.4 实现上的取舍

在项目初期，我原本也希望比较两种 RBF 求解方式在控制点数量增加时的建模时间差异，并与 IDW 这类“查询时加权”的方法作对照。但由于本次作业时间有限，后续并没有继续深入做系统性的时间测试。因此本文虽然保留了 RBF_AffineFirst 与 RBF_Full 两个接口，但实验分析主要仍集中在变形效果与方法结构理解上，而没有进一步展开求解时间方面的定量比较。

5.5 小结

相较于 IDW，RBF 更像是先拟合一张整体连续变形场，再对任意点进行统一求值。它在形式上更全局，也更适合写成完整的线性系统。但与此同时，RBF 的系数求解与控制点整体分布联系更紧密，参数和数值行为也相对更全局耦合。

在本文实现中，我将其保留为两种求解接口，主要是为了在同一程序框架下对比不同建模路线，而不是试图在本次作业中对其做更深入的理论扩展。

6 前向映射、Mask 与补洞实现

6.1 前向映射带来的空洞问题

本文的图像变形采用前向映射 (forward warping) 实现。也就是说, 对原图中每个像素 (x, y) , 先通过 warper 计算其目标位置

$$(x', y') = f(x, y),$$

再将该像素颜色写入结果图中。

这种做法实现直观, 但也带来一个典型问题: 目标图像是离散网格, 连续平面上的映射结果在取整后不可能保证每个目标像素都恰好被命中。因此, 只要局部拉伸或压缩较明显, 就会出现空洞或裂缝。

6.2 两类 mask 的作用

为了更准确地区分“真正需要补的洞”和“本来就不该有内容的背景”, 本文在 warping 阶段维护了两张 mask:

- **written_mask**: 记录哪些目标像素已经被真实写入过颜色;
- **coverage_mask**: 记录哪些区域在连续几何意义上本来应当属于 warped 图像覆盖范围。

其中, **written_mask** 较容易理解: 若某个目标位置被某个原图像素成功写入, 则其位置为 1, 否则为 0。

而 **coverage_mask** 的构造更关键。程序将原图像的每个网格单元拆成两个三角形, 对其四个顶点经 warper 变换后的结果做光栅化, 将这些三角形覆盖到的目标像素标记为 1。这样得到的 **coverage_mask** 描述的是“理论上应该有图像覆盖”的区域。

因此, 一个点若满足:

$$\text{coverage_mask} = 1, \quad \text{written_mask} = 0,$$

它才是合理的空洞候选。这样可以有效避免把图像外部背景误判为待补区域。

6.3 ANN 风格的局部补洞

当前补洞模块采用一种较简化的 ANN/KNN 风格局部近邻插值。它的思路是: 对于待补像素, 从当前已知像素样本中搜索一定范围内的若干近邻, 再按反距离权重对其颜色做加权平均。

设待补点为 (x, y) , 若邻域中找到样本点 (x_j, y_j) , 则其距离平方记为

$$d_j^2 = (x - x_j)^2 + (y - y_j)^2,$$

采用权重

$$w_j = \frac{1}{d_j^2 + \varepsilon}$$

计算颜色平均。若恰好出现零距离命中，则直接复制对应颜色。当前实现中，每轮补洞使用最近的若干个样本点，并迭代进行多轮传播。

这里的 ANN 实现并不是本文关注的重点，因此保持了较简洁的局部近邻搜索写法，没有进一步引入更复杂的数据结构。

6.4 范围约束与边界保护

相比 ANN 本身，本文认为更关键的是补洞**只在合理范围内进行**，而不是对整张图像一概扩散。

程序在判定某个像素是否可补时，主要检查两层条件：

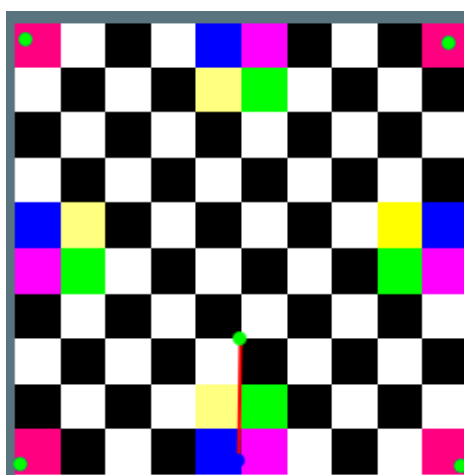
(1) **Coverage 约束** 该点必须位于 `coverage_mask` 内部，且在一个很小的邻域里有足够多的 `coverage` 像素。若 `coverage` 太稀疏，则更可能是 `warped` 图像边缘或外部背景，此时不补。

(2) **Known 邻域约束** 该点附近必须已有足够多的已知样本点。这样可以避免算法隔着很远去硬性猜测颜色，使补洞更适合处理前向映射产生的小裂缝和小孔洞，而不是大面积的图像修复。

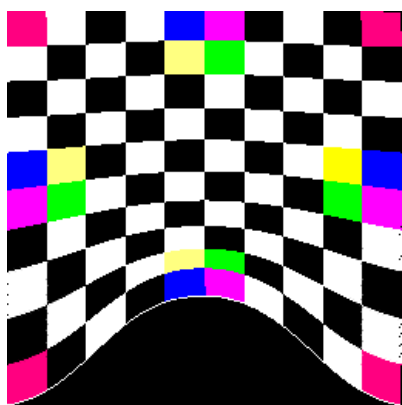
这种双重约束使得当前补洞结果虽然不追求复杂的图像修复能力，但比较适合本文的实际需求：**不乱补，不越界，只在应该属于 `warped` 图像内部、且周围已有足够样本的地方补小洞。**

7 实验结果

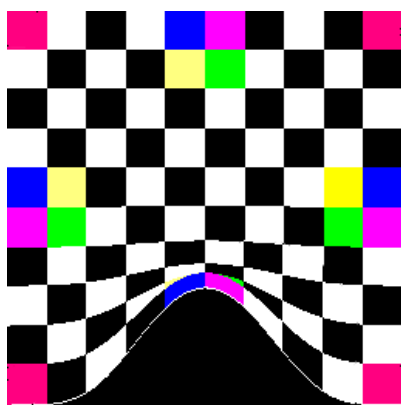
1. 不同 warping 方法在局部控制上的表现；
2. 同样控制点对下，变形场的平滑性与整体观感；
3. 在前向映射框架下，补洞模块对结果可读性的改善；
4. 自定义改进方法是否能够减少对密集控制点的依赖。



(a) Control Points



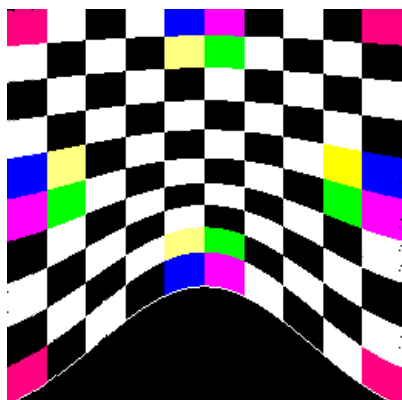
(b) IDW



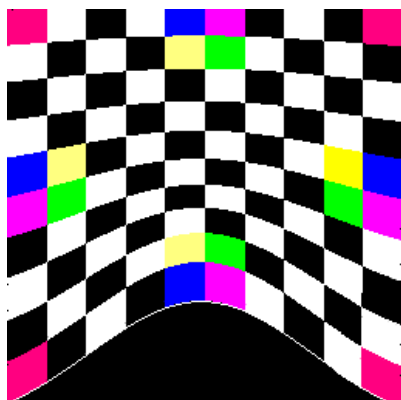
(c) IDW_A



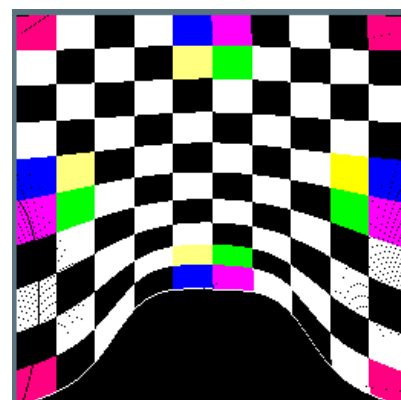
(d) IDW_R



(e) RBF_A



(f) RBF_F

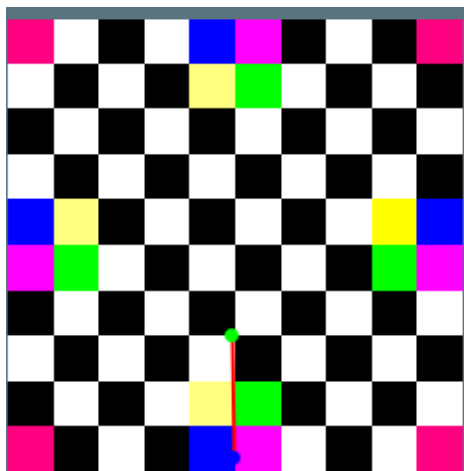


(g) IDW_diff_cor

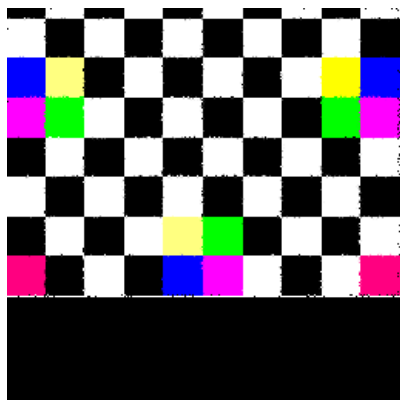
$$\sigma \text{ replaced by: } y = \frac{1}{(x^2 + c^2)^{\frac{1}{2}}} \frac{1}{\left(1 + \frac{x^2}{a^2}\right)}$$

$$(c = 0.1, a = 10)$$

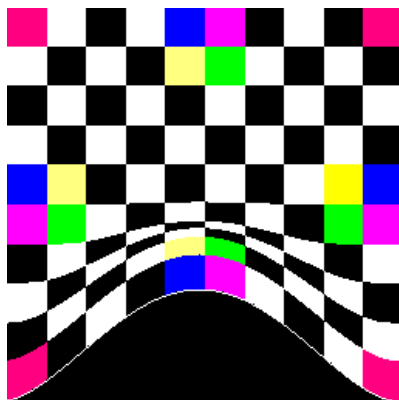
图 1: 通过变形效果观察不同核函数的形状特征



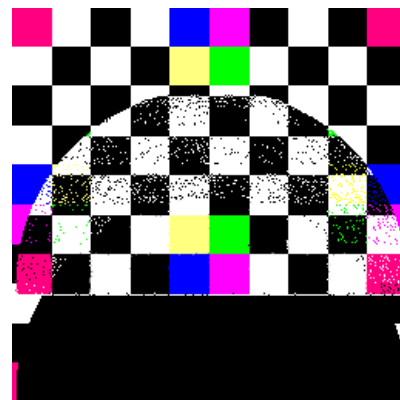
(a) Control Points



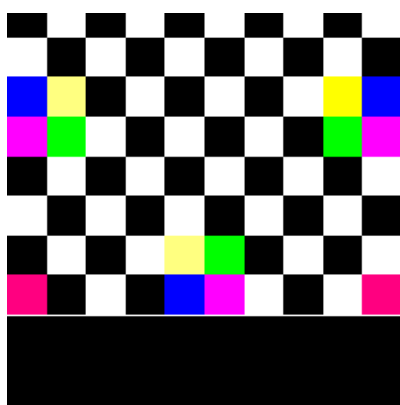
(b) IDW



(c) IDW_A

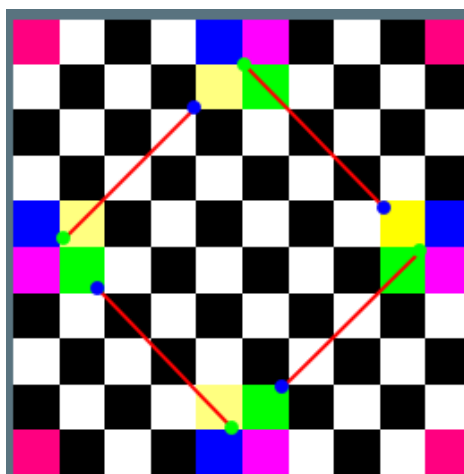


(d) IDW_R

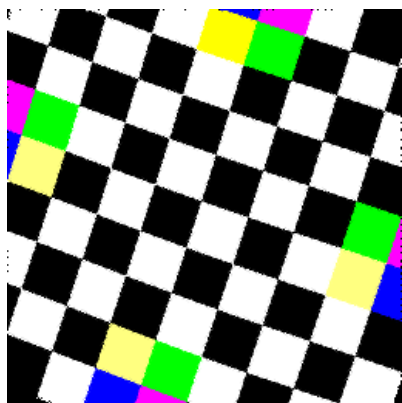


(e) RBF_A

图 2: 单点测试



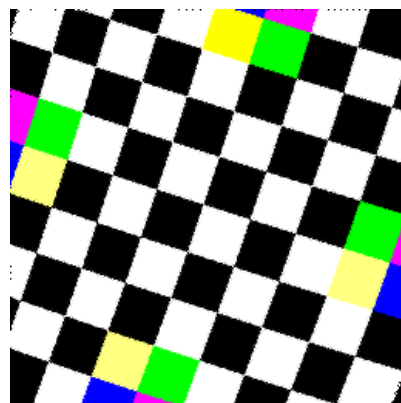
(a) Control Points



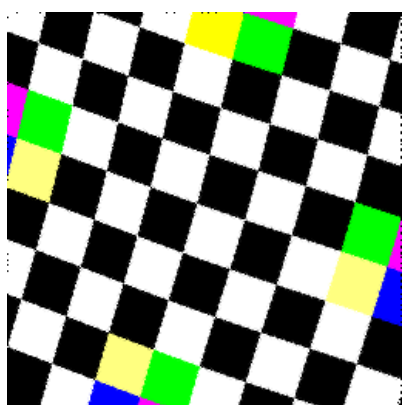
(b) IDW



(c) IDW_A

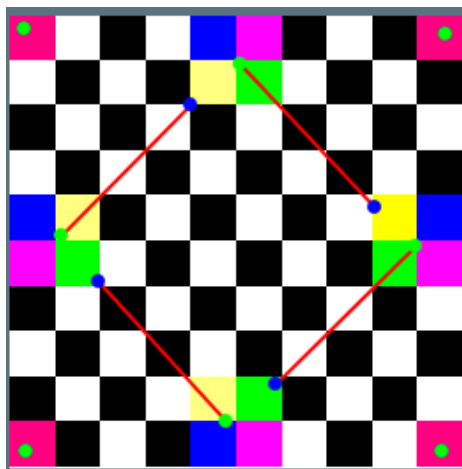


(d) IDW_R

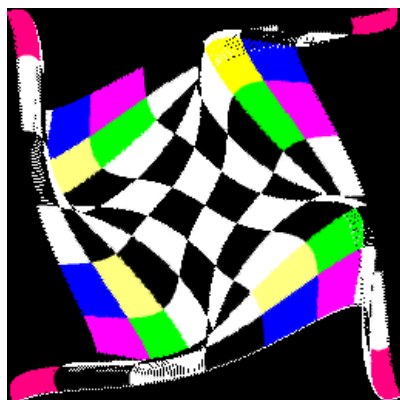


(e) RBF_A

图 3: 不固定边界的扭曲



(a) Control Points



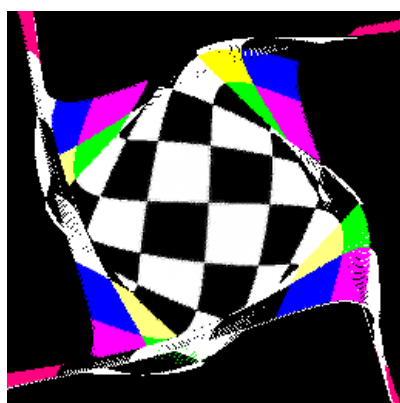
(b) IDW



(c) IDW_A



(d) IDW_R



(e) RBF_A

图 4: 固定边界的扭曲